

Loss Functions, Likelihoods, and Neural Networks

In this document, you'll find a basic overview of some of the ideas explored in the tutorial. In this tutorial, we explore modeling approaches to a dataset which contains bike rentals counts over two years. The counts were measured along with a set of features that includes the hour, day, weather, workingday (binary), and others. We will mainly restrict our attention to modeling the average counts per hour for each day of the week, adding in more features as we go.

There are two python files - a quick exploratory data analysis (EDA), and the main modeling file. The EDA notebook will inform us what sorts of models might be applicable. Don't spend too much time on this - the goal is to get some basic familiarity with the dataset. In particular, we'll get an idea of the following:

- the functional form of the average counts per hour, day
- summary statistics vs hour for each day
- histograms of counts vs various features

Following this, we begin modeling in the Modeling notebook. This notebook contains more discussion of the models used, extensions, and other methods. It explores two model families - linear regression models and Poisson regression models. We begin with simple setups, then add complexity such as non-linear terms, effects of other features, and a 'fully' nonlinear treatment using feed-forward neural networks. An interesting bonus is at the end - a recurrent neural network can also be used.

In the remainder of this document, we will give some basis for the methods, techniques, and concepts explored in the tutorial.

0.1 Likelihoods and Maximum Likelihood Estimation

One of the ideas we want to motivate is a connection between the choice of loss function and an associated underlying probability model. In some sense, this is an equivalence. Let's start with OLS which we take to be minimizing the mean-square error (MSE):

$$\text{MSE}(x, \mu) = \frac{1}{2n} \sum_{i=1}^n (x_i - \hat{\mu}_i)^2 \quad (1)$$

where $\hat{\mu}$ is the thing we're estimating - typically the population mean.

Now, consider instead we model the following:

$$y = f(x) + \epsilon \quad \text{where } \epsilon \sim \mathcal{N}(0, \sigma^2) \quad (2)$$

where σ^2 is constant. This is the standard prescription for linear regression, $f(x) = x^T \beta$. There is an equivalent form of this statement:

$$y - f(x) \sim \mathcal{N}(0, \sigma^2) \quad (3)$$

$$\rightsquigarrow p(y | f, x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left\{ -\frac{(y - f(x))^2}{2\sigma^2} \right\} \quad (4)$$

We now have a model of the conditional probability, $p(y | f, x)$, of the response y given the function f and the covariates/features x . A natural way to use this probability is to infer the function f . Let's take $f(x) = x^T \beta$ where the parameter(s) β are what we'd like to infer:

$$p(y | \beta, x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left\{ -\frac{(y - x^T \beta)^2}{2\sigma^2} \right\} \quad (5)$$

A natural question is as follows: for what values of β is the observation of y , given the features x , the most likely? This is known as maximum likelihood estimation (MLE). In order, we will maximize $p(y | \beta, x)$ given our data, $\mathcal{D} = \{x^{(i)}, y^{(i)}\}_{i=1}^n$. We construct the probability of our data (assuming independent measurements/observations) - we will refer to this as the likelihood:

$$p(\mathcal{D} | \beta) = \prod_{i=1}^n p(y^{(i)} | \beta, x^{(i)}) \quad (6)$$

$$= \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right)^n \exp \left\{ -\frac{1}{2\sigma^2} \sum_{i=1}^n \left(y^{(i)} - \beta^T x^{(i)} \right)^2 \right\} \quad (7)$$

We can simplify things by instead maximizing the log-likelihood (it'll have the same solution):

$$l(\mathcal{D} | \beta) = -\frac{1}{2\sigma^2} \sum_{i=1}^n \left(y^{(i)} - \beta^T x^{(i)} \right)^2 + \text{constant} \quad (8)$$

Assuming that σ is fixed, it doesn't affect optimization. In summary, to infer β using maximum likelihood, we minimize the negative log-likelihood (NLL):

$$L(\beta) = \frac{1}{2\sigma^2} \sum_{i=1}^n \left(y^{(i)} - \beta^T x^{(i)} \right)^2 \quad (9)$$

This is then seen to be equivalent to minimizing the MSE. Thus, the relation between MSE and a Gaussian error (used for inference). PyTorch implements NLL directly for a given probability output, and also for various distributions, such as the Poisson distribution which we explore next.

A tangential note: cross-entropy is equivalent to MLE for logistic/multinomial regression which better illuminates how neural networks perform classification.

0.2 The Poisson Distribution, Process, and Regression

The main points to glean from this are

- Poisson distribution is natural for count data
- It is derivable under assumptions (skim the derivations)
- A natural relation between features and the Poisson mean/rate is $\ln \lambda = X^T \beta$
- To perform Poisson regression, we can perform MLE using the Poisson's log-likelihood.

0.2.1 Deriving the Poisson Distribution

Consider modeling arrival times (counting processes), such decay particles hitting a detector or goals scores by a given soccer player. For our tutorial, consider bike rentals. Take $N(t)$ to be the number of events occurring at or before time t , and $P_k(t) = \mathbb{P}(N(t) = k)$ to be the probability of k counts within the interval of length t . Let us place some assumptions on this random variable, $N(t)$.

1. Independent increments: the number of events in disjoint time intervals are independent.
2. Stationary increments: $P_1(\Delta t) \approx \lambda \Delta t$ - probability depends only on interval length (linearly here).
3. No simultaneous events: $P_n(\Delta t) \approx 0$ for $k > 1$.

We will derive the resulting distribution, the Poisson distribution, multiple ways.

via the binomial:

Split the time interval t into n segments of lengths $\Delta t = t/n$ so that Δt is sufficiently small so as to ignore multiple events. By the assumptions, the number of events in each segment is independent. The probability of an events occurring in segment i is then simply $\lambda \Delta t$, for all i . In determining the probability of k total events, we are then lead to a product of n identical Bernoulli distributions, or so a binomial distribution

$$P_k(t) \approx \binom{n}{k} (\lambda \Delta t)^k (1 - \lambda \Delta t)^{n-k} \quad (10)$$

$$P_k(t) = \lim_{n \rightarrow \infty} \binom{n}{k} \left(\frac{\lambda t}{n} \right)^k \left(1 - \frac{\lambda t}{n} \right)^{n-k} \quad (11)$$

In taking the limit $n \rightarrow \infty$, we see the following using common properties and the so-called rising/falling notation for binomial coefficients

$$\binom{n}{k} = \frac{\prod_{i=0}^{k-1} (n-i)}{k!} = \frac{n^k}{k!} \prod_{i=0}^{k-1} \left(1 - \frac{i}{n} \right) \rightarrow \frac{n^k}{k!} \quad (12)$$

and so

$$\left(1 - \frac{\lambda t}{n} \right)^{n-k} \rightarrow e^{-\lambda t} \quad (13)$$

$$\binom{n}{k} \left(\frac{\lambda t}{n} \right)^k \rightarrow \frac{(\lambda t)^k}{k!} \quad (14)$$

All in all we have

$$P_k(t) = e^{-\lambda t} \frac{(\lambda t)^k}{k!} \quad (15)$$

an ODE (Kolmogorov forward equations):

Given the earlier discussion of features of $P_k(t)$, we can obtain a recursion relation by conditioning on the last time step (its also simple enough to intuit, but this trick is cool)

$$P_k(t + \Delta t) = \sum_s \mathbb{P}(N(t) = k - s | N(\Delta t) = s) \mathbb{P}(N(t) = s) \quad (16)$$

$$= \sum_{s=0}^1 \mathbb{P}(N(t) = k - s) \mathbb{P}(N(t) = s) \quad (17)$$

$$= P_k(t)(1 - \lambda\Delta t) + P_{k-1}(t)\lambda\Delta t \quad (18)$$

where above we used the fact of independent increments and the absence of simultaneous events. This equation becomes

$$\frac{\partial P_k(t)}{\partial t} = -\lambda P_k(t) + \lambda P_{k-1}(t) \quad (19)$$

$$P_k(t) = \lambda e^{-\lambda t} \int dt e^{\lambda t} P_{k-1}(t) \quad (20)$$

For the special case of $P_0(t)$ we find

$$P_0(t + \Delta t) = P_0(t)(1 - \lambda\Delta t) \quad (21)$$

$$\frac{\partial P_0(t)}{\partial t} = -\lambda P_0(t) \quad (22)$$

$$\rightsquigarrow P_0(t) = e^{-\lambda t} \quad (23)$$

Now to find $P_1(t)$:

$$\frac{\partial P_1(t)}{\partial t} = -\lambda P_1(t) + \lambda e^{-\lambda t} \quad (24)$$

$$(P_1(t)e^{\lambda t})' = \lambda \quad (25)$$

$$\rightsquigarrow P_1(t) = \lambda t e^{-\lambda t} \quad (26)$$

Deriving higher orders are straight-forward, yielding

$$P_k(t) = \frac{(\lambda t)^k}{k!} e^{-\lambda t} \quad (27)$$

0.2.2 The Poisson Process and Poisson Regression

For the sake of completeness, we give the formal definition of a Poisson process below:

Definition. (Following Ross - Introduction to Probability Models) The counting process $N(t)$ is said to be a Poisson process having rate $\lambda > 0$ if

1. $N(0) = 0$
2. The process has independent increments (the counts for non-overlapping intervals are independent).
3. The number of evenets in any interval of lnegth t is Posson distributed with mean λt . That is, $\forall t, s \geq 0$

$$P\{N(t+s) - N(s) = n\} = e^{-\lambda t} \frac{(\lambda t)^n}{n!}$$

or so has stationary increments (distribution depends only on interval length).

The basic jist

Moving the focus back to modeling smethnig like bike rental data, take the response Y to be some count data which we suspect follows a Poisson distribuion. We summarize basic features below (the depedence of λ on time is buried in λ for now):

$$P(Y = y) = \frac{\lambda^y}{y!} e^{-\lambda} \quad (28)$$

$$\mu = \mathbb{E}[Y] = \lambda \quad (29)$$

$$\text{Var}(Y) = \lambda \quad (30)$$

The dependence of the variance on the mean may be an issue. We need to compare the data's variance with its mean to ensure this is a reasonable model.

A bit about the Negative Binomial distribution - SKIPABLE

In the case of overdispersion, $\text{Var}(Y) > \mu$, we can use a negative binomial model. The negative binomial can be understood as a hierarchical model with (see Casella & Berger)

$$Y \mid \lambda \sim \text{Poisson}(\lambda) \quad (31)$$

$$\lambda \sim \text{Gamma}(\alpha, \beta) \quad (32)$$

where the gamma pdf is given as (taking β as the scale, but rate $\sim 1/\text{scale}$)

$$f(\lambda \mid \alpha, \beta) = \frac{\lambda^{\alpha-1} e^{-\lambda/\beta}}{\Gamma(\alpha) \beta^\alpha} \quad (33)$$

The resulting marginal distribution for the count is a negative binomial

$$f(y \mid \alpha, \beta) \sim \text{Neg Binomial}(\alpha, \frac{1}{1+\beta}) \quad (34)$$

$$= \binom{y + \alpha - 1}{y} \left(\frac{1}{1+\beta} \right)^\alpha \left(1 - \frac{1}{1+\beta} \right)^y \quad (35)$$

Using the gamma prior has the effect of regulating the variance. In terms of α and β , we have the mean

$$\mathbb{E}[Y] = \mathbb{E}[\mathbb{E}[Y \mid \lambda]] \quad (36)$$

$$= \mathbb{E}[\lambda] \quad (37)$$

$$= \alpha\beta \text{ since } \lambda \sim \Gamma(\alpha, \beta) \quad (38)$$

and the variance

$$\text{Var}(Y) = \mathbb{E}[\text{Var}(Y \mid \lambda)] + \text{Var}(\mathbb{E}[Y \mid \lambda]) \quad (39)$$

$$= \mathbb{E}[\lambda] + \text{Var}(\lambda) \quad (40)$$

$$= \alpha\beta + \alpha\beta^2 \text{ since } \lambda \sim \Gamma(\alpha, \beta) \quad (41)$$

$$= \mu + \mu^2/\alpha \quad (42)$$

This allows to model to learn a more variable variance, $\alpha \rightarrow \infty$ if Poisson and $\alpha \rightarrow 0$ if heavily overdispersed.

Poisson Regression

For applications, we have some sample of features X which impact the count data, particularly the poisson distribution parameter λ . We'd like to find a "natural" way to relate our features X to the rate λ . We write this as $g(\lambda) = X^T \beta$ where g is a "link function". For linear regression, we want (typically) μ which we model as $\mu = X^T \beta$ - so the identity link is used. The more general notion we find ourselves discussing is known as a generalized linear model (GLM).

What follows is a bit technical, a bit slippery to me, but has some natural intuition. First, the intuition: the rate λ is a positive quantity, so the model should respect that. Taking $\lambda = X^T \beta$ would likely result in negative counts; however, taking $\lambda = \exp X^T \beta$ would keep the counts positive. This is enough to jump to the regression section. A bit of the flavor of GLMs is added below (very rough/loose).

quick and dirty sketch of the GLM approach

The general prescription of generalized linear models: we match the pdf with its exponential family parameterization. It is straight-forward to express the Poisson distribution as an exponential:

$$P(Y = y) = \frac{\lambda^y}{y!} e^{-\lambda} \quad (43)$$

$$P(Y = y) = \exp(y \ln \lambda - \ln y! - \lambda) \quad (44)$$

Since y , being the sample count, is our *sufficient statistic*, we select $\ln \lambda$ to be our linear predictor:

$$\ln \lambda = \beta^T X \quad (45)$$

Regression as MLE with Poisson Likelihood

To perform regression, we now use the log-likelihood of the Poisson distribution together with our parametrization of the rate, $\ln \lambda = \beta^T X$:

$$l(y|\beta) = \sum_i \left(y^{(i)} \ln \lambda - \ln y^{(i)}! - \lambda \right) \quad (46)$$

$$= \sum_i \left(y^{(i)} \beta^T X^{(i)} - \ln y^{(i)}! - \exp\{\beta^T X^{(i)}\} \right) \quad (47)$$

and perform maximum likelihood estimation using gradient descent, stochastic gradient descent, or whatever algorithm you'd like.

Homogeneous Poisson Process Regression and Offsets

Now, we about if the rate (intensity) varies, say it depends on time. A Poisson process has a homogeneous rate, $\lambda(t) = \lambda t$. We could use the same prescription $\ln \lambda(t) = \beta^T X$; however, this result is an exponential time dependence since time is one of our features. To handle this, we can use a so-called offset,

$$\ln \lambda(t) = \ln(t) + X^T \beta \quad (48)$$

where the effects of time are built in. The other features are modeled as in a regular poisson regression.

Nonhomogeneous Poisson Process Regression and One-hot Encoding

We can also take the rate to be non-homogeneous (a non-homogeneous Poisson process) where the rate is a piece-wise function of time:

$$\lambda(t) = \sum_t \lambda_t 1_t$$

where we take t to be discrete/binning and one-hot encoded. In other words, we model our time intervals, for the bike rental count is the hour, as separate features - each hour/time-interval gets its own rate/coefficient. This is equivalent to the statement that the model no longer treats all time the same.

We can additionally do this for other variables, such as the day, so that each day gets its own independent rate.

0.3 Remarks and Extensions

- In the tutorial, will we be able to use the same one-hot encoding method for linear regression as well.
- We can replace $\ln \lambda = X^T \beta$ with $\ln \lambda = NN(X)$ where NN is a neural network. This extends our setup to nonlinear models.
- It's possible that a MSE loss could attain the same estimate as a Poisson loss....